

一、选题说明

1. 基于大语言模型的软件工程智能体

1.1 项目目标

在当今软件工程领域，利用大语言模型辅助软件开发已成为新一代工程实践的核心趋势。然而，通用大模型在复杂、多阶段软件项目中的自主性与稳定性仍有较大提升空间。本课程设计旨在引导学生设计并实现一个**面向软件项目开发的智能体系统**，使其能够在一个或多个连续开发阶段中辅助或自动完成典型软件工程任务。

具体目标包括：

- 理解大语言模型的基本原理及其在软件工程中的应用范式；
- 掌握智能体系统的设计方法，包括任务分解、上下文管理、工具调用等关键技术；
- 实现一个可运行的智能体原型系统，支持**需求分析、系统设计、编码实现、测试、调试、文档编写**中的至少一个连续阶段组合（详见1.2）；
- 通过系统化的测试与评估，验证智能体在基准任务上的有效性；

各小组可在上述总体目标框架下，**自主选择智能体所覆盖的具体功能范围**（最小组合见1.2），鼓励结合兴趣或实际项目背景进行创新。

1.2 技术指标与交付物

1.2.1 功能要求

A. 该智能体应支持软件工程过程中**几个连续阶段**，开发团队必须明确定义其智能体的**输入形式**（如自然语言需求文档、PRD、用户故事、UML模型、代码片段、错误日志等）与**输出产物**（如设计模型、代码文件、测试用例、修复补丁、使用说明等）。可选的最小功能组合包括：

组合编号	覆盖阶段	具体要求
a	分析 + 设计	根据系统需求或PRD，自动生成系统分析与设计模型，包括但不限于 类图、活动图、状态机图 中的至少两种，输出可采用文本描述、
b	实现 + 测试	根据需求、PRD或设计模型，自动生成可运行的 程序代码 （指定语言，如Python/Java）及对应的 单元测试用例 ，并能执行测试并汇
c	调试 + 程序修复	给定错误的程序及触发错误的测试用例，自动 定位错误位置 ，解释错误原因，并生成 修复后的正确代码 。

小组也可以选择覆盖更多阶段（如分析+设计+实现）。

1.2.2 非功能与集成要求

B. **命令行启动**：智能体应能够通过命令行接口（CLI）以非交互或半交互方式启动，自动完成所定义的任务，并输出结果到指定目录。例如（但不限于）：

```
./agent --task design --input requirements.md --output design/
```

C. **IDE集成**：智能体应能够集成到主流IDE中（如VSCode、Eclipse、IntelliJ IDEA等），至少实现以下一种集成形式：

- 作为插件或扩展，提供菜单/快捷键触发；
- 通过语言服务器协议（LSP）或自定义命令与IDE交互；
- 提供可嵌入IDE的Web服务接口。

D. **基准测试**：智能体必须通过指定的基准测试，以证明其在所选任务上的有效性。基准测试由课程组提供或小组自行构建但需经教师认可。测试内容应覆盖：

- 至少5个不同场景/案例；
- 度量并报告成功率。

1.2.3 交付物清单

开发小组需要提交以下文档（均要求PDF格式，并附源代码仓库链接）：

编号	文档名称	主要内容要求
a	智能体工作原理技术报告	包括大模型选择（如GPT、Claude、Llama等）、提示工程策略、任务分解与编排机制、错误处理与重试逻辑、工
b	智能体系统设计方案	系统架构图、模块划分、数据流、核心类/接口设计、关键技术选型理由。
c	智能体系统测试方案与测试报告	测试环境、测试用例设计（正常/异常/边界）、基准测试结果、性能分析、局限性说明。
d	智能体使用说明与应用案例	安装配置步骤、CLI命令详解、IDE集成方法、至少两个完整应用案例（从输入到输出的演示）。

1.3 开发约束

A. **禁止直接扩展已有软件工程智能体**：不得基于MetaGPT、ChatDev、AutoGPT等现有软件工程智能体系统的代码进行简单封装或参数调整。但允许使用**通用智能体框架**作为底层基础设施，例如：

- **CrewAI**：用于定义角色、任务和流程编排；
- **LangGraph / LangChain**：用于构建基于大模型的图式工作流；
- **Semantic Kernel、DSPy** 等研究性框架。

若使用了上述框架，必须在技术报告中明确说明哪些部分为自主实现，哪些由框架提供。

B. **代码与数据开源**：所有实现的源代码、配置文件、测试数据必须提交至Git仓库（校内GitLab），仓库需包含详细的README，说明如何复现环境与运行智能体。

C. **里程碑检查点**：

- 第1周：选题确认
- 第3周：需求分析（明确所选功能组合、基准测试方案）与系统设计；
- 第6周：设计方案、原型系统实现、确定测试方案；
- 第8周：最终交付（所有文档+演示）。

2. SysML2 仿真引擎原型

2.1 项目目标

SysML2（系统建模语言）是系统工程的标准化建模语言的第二版，它支持更精确的系统行为描述与分析。在复杂系统（如嵌入式系统、机器人、航空航天系统）的设计过程中，为了验证系统行为，需要一种能够执行 SysML2 模型的仿真引擎。本项目的目标是**设计并实现一个 SysML2 仿真引擎的原型系统**，能够解析 SysML2 模型中的关键行为元素（如动作、状态机、序列流），并在离散事件仿真环境中执行模型，输出仿真轨迹与验证结果。选择此题目的小组需要完成从模型解析到仿真执行的核心链路。

具体目标包括：

- 理解 SysML2的核心语法与语义；
- 掌握基于模型的系统工程（MBSE）与仿真技术；
- 基于MOKA实现一个可扩展的仿真执行框架（MOKA是一种UML仿真引擎）；
- 通过典型系统案例验证仿真引擎的正确性与实用性。

2.2 技术指标与交付物

2.2.1 功能要求

A. 该仿真引擎应支持以下 SysML2 行为模型的**子集**（最小可行功能），小组可明确所选子集：

- **动作**：基本动作（赋值、运算）、条件分支、循环，；
- **状态机**：状态、转换、触发事件（支持信号/定时事件）。

- B. 该仿真引擎应支持必要的SysML2 表达式计算。
- C. 引擎应具备以下核心能力：
- **模型解析**：利用官方解析器读取 SysML2 模型，构建内部模型抽象语法模型。
 - **模型转换**：将SysML2模型转换成MOKA支持的UML模型。
 - **仿真执行**：利用MOKA引擎实现SysML2仿真执行。
 - **结果输出**：生成仿真日志（时间、状态、事件、变量值），并可导出标准输出。
- D. 引擎应提供命令行使用方式，例如（但不限于）：`simulate --model model.sysml2 --steps 100` 或 `--until-condition "x>10"`；

2.2.2 非功能与集成要求

- D. **模型限制**：初始版本不要求支持全部 SysML2 模型元素，但须明确支持的功能子集与不支持的边界。
- E **基准测试**：提供至少 5 个测试模型（难度递增），通过对比预期行为轨迹（由教师提供或小组手动验证）来验证引擎正确性。仿真结果应与参考输出一致。

2.2.3 交付物清单

编号	文档名称	主要内容
a	仿真引擎技术报告	总体架构、支持的 SysML2 子集说明、模型转换规则。
b	系统设计与实现文档	模块划分、核心类图、关键算法伪代码等。
c	测试方案与测试报告	测试策略（单元测试/集成测试）、测试用例（每个测试模型的目标与预期输出）、实际仿真结果对比、结果分析。

2.3 开发约束

- A. **基于MOKA仿真引擎**：仿真引擎后端必须基于MOKA。
- B. **编程语言与环境**：项目必须使用Java完成。
- C. **里程碑检查点**：
- 第1周：确定选题；
 - 第3周：相关技术调研（SysMLv2、UML、MOKA、模型转换等）和需求分析。
 - 第6周：系统设计、原型实现、测试方案；
 - 第8周：交付验收。

2.4 参考资料

- SysML v2规范：<https://www.omg.org/sysml/sysmlv2/>
- SysML v2 Pilot Implementation：<https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation>
- Papyrus项目（含MOKA）：<https://eclipse.dev/papyrus/>

3. 基于 CRDT 的支持隐私保护的结构化数据协同编辑器

3.1 项目目标

在实时协同编辑系统中，多用户可同时修改共享数据，无冲突复制数据类型（CRDT）因其无需中心服务器即可保证最终一致性的特点，成为新一代协同编辑的基础技术。然而，传统 CRDT 通常假定所有参与者拥有相同的数据视图，无法满足隐私保护需求——不同用户应当只能看到与其权限相关的数据子集（例如树形文档中的特定分支、表格中的某些行列）。

本项目旨在设计并实现一个基于 CRDT 的协同编辑器原型，支持结构化数据（如树、列表、JSON 对象）的多用户实时协同，并能够通过数据变换（例如双向变换 BX）实现可配置的隐私保护视图。核心思想是：底层使用 CRDT 存储完整数据，在发送给不同用户之前，利用双射变换（或类似机制）将完整数据映射为视图数据，从而对用户屏蔽其无权访问的部分。用户的编辑操作经逆变换映射回完整数据，再通过 CRDT 同步到其他节点。

选择此题目的小组需要完成以下任务：

- 理解 CRDT 基本原理，实现一种支持树或列表结构的 CRDT（如 JSON CRDT、RGA、TreeCRDT）；
- 设计并实现一种数据视图变换机制（基于 BX 或更轻量的投影/加密方案），使得不同用户看到的数据结构不同；

- 构建一个多用户协同编辑原型系统（可以是命令行模拟或简易 Web 界面），验证隐私保护下的正确协同。

通过本项目，学生将掌握分布式一致性、协同编辑、访问控制及函数式数据变换等核心概念。

3.2 技术指标与交付物

3.2.1 功能要求

A. CRDT 协同核心：

- 支持结构化数据模型（至少支持**树结构**，例如 XML / JSON 树）。每个节点可以包含键值对或文本内容。
- 实现 CRDT 合并算法，保证多用户并发编辑后**最终一致性**（无冲突，不丢失操作）。
- 支持以下基本编辑操作：插入节点、删除节点、修改节点内容、移动节点（可选）。

B. 隐私保护视图：

- 定义并实现视图构建机制，如 **双射变换 (BX)** 语言，将完整 CRDT 数据转换为用户视图数据（正变换）。
 - 当用户编辑视图数据后，系统应将视图修改转换回完整数据上的操作（逆变换）。
- 不同用户登录后只能看到自己的视图，但所有用户的编辑最终通过 CRDT 在完整数据层合并。
- 至少实现两种不同的隐私策略（例如：按用户角色隐藏特定标签；基于节点属性的访问控制）。

C. 协同编辑器前端/接口：

- 提供简单的文本或图形化界面（可以是 Web 页面），展示当前用户的视图树。
- 支持基本的编辑命令（添加子节点、删除、重命名等）。

3.2.2 非功能与集成要求

D. **网络与同步**：必须保证 CRDT 合并的正确性，不依赖总顺序。允许用户在编辑数据时断网操作，并在连接网络后同步数据。

3.2.3 交付物清单

编号	文档名称	主要内容
a	协同编辑器工作原理技术报告	CRDT 数据结构与合并算法；隐私保护机制（BX 变换设计）；操作传播与合并流程；一致性证明思路。
b	系统设计方案	整体架构（客户端-服务器/去中心化）、模块划分（CRDT 引擎、视图变换层、前端渲染）、核心类图、关键算法等。
c	测试方案与测试报告	单元测试（CRDT 合并、变换函数）、场景测试（并发冲突、隐私策略切换）、结果分析、已知局限性。
d	使用说明与应用案例	安装运行指南、权限配置文件格式、至少两个完整案例（如：协同编辑任务清单、多角色项目文档树）。

3.3 开发约束

A. **强制使用 CRDT**：必须基于 CRDT 实现协同一致性。可以采用已知 CRDT 论文中的算法（如 RGA、Logoot、Auctus、JSON CRDT），可参考现成 CRDT 库（如 yjs 、 automerge ）。

B. **隐私变换方式**：鼓励采用双向变换（BX）理论，但也可以使用其他等价的**可逆投影**机制。必须在技术报告中清晰定义变换的数学性质。

C. 里程碑检查点：

- 第1周：确定选题；
- 第3周：需求分析、相关技术调研、系统设计；
- 第6周：原型实现、测试方案；
- 第8周：交付验收。

二、过程要求

- 软件项目必须托管到指定Gitlab平台
- （如果有必要）开发团队应使用华为云提供的云服务器或大模型资源完成项目
- 鼓励使用华为CodeArts Agent进行开发

4. 开发团队必须定期进行代码质量扫描

三、评价指标

说明：下面的评价指标对应课程的五个教学目标

- **软件系统（40%）**：需求达成度+完成质量
 - 软件分析和设计的质量
 - 代码质量将占有分数（占5%）
 - 课程将设置一次中期检查（课程第5~6周，占10%），中期检查将直接影响最终得分
- **开发环境（10%）**：使用现代开发环境和工具开发，包括但不限于
 - 能够熟练使用git系统进行代码管理，包括分支、提交、pull request等
 - 能够使用wiki、bug issue等功能管理项目开发
 - 能够使用代码分析、自动化编译和测试功能改进开发
 - 使用Docker等工具规范开发环境
- **团队合作（10%）**：有合理分工并且能够相互协作
 - 分工明确性
 - 协作程度
- **文档和报告（30%）**：最终文档和演示的质量
 - 文档的格式规范性和内容的规范性（15%）
 - 演示汇报（10%）：展示软件工程全过程，重点阐述难点及解决方案
 - 技术学习（5%）：对相关技术的学习过程及收获展示
- **过程管理（10%）**：有合理的进度安排并能充分利用时间、把握进度
 - 有开发计划和明确的里程碑（里程碑应对应需求）
 - 开发应充分利用时间
 - 能够基本按照计划完成开发

四、课程安排

1. 组队

每个开发团队由4-6名成员组成，其中1名作为组长。

- 组长需要负责组织和管理项目开发，协调小组成员、解决成员间的分歧。组长将在最终评分中获得组长加成（+5%~+10%）。
- 小组成员需要服从组长安排，对于拒不服从安排的成员，经组长申请、助教核实后，在最终评分中将受到惩罚（-5%~-50%）。
- 开发团队规模小于6人时，将有机会获得额外的加分。

2. 选题

每个小组在上课之前需要对所有选题进行研究，并在第一次上课时确定选题。每个小组只能选择一个题目，且不建议在进行过程中更换题目，因此请谨慎进行选题可行性分析。

每个选题包括若干需求，选题需求将有可能在课程后半段（课程第5~6周）发生一次演化。

在明确规定的需求之上，各小组可根据情况自行扩展需求，当扩展需求比原需求显著增强时，可获得额外加分。

3. 组长会

每次上课将组织组长会。组长会以选题为单位进行组织，所有相同选题的小组长负责参加讨论。组长会的内容如下：

- 每个组长进行每周工作小结
- 汇总各小组遇到的问题
- 相互讨论，以便形成解决方案

组长会是各小组相互交流和学习的机会，也是共同商讨复杂问题的解决方案的机会。各小组应充分利用这一机会，在竞争中相互帮助。

4. 技术方案保护制度

为了鼓励和保障在组长会上分享解决方案和思路小组，各小组可以申请“技术方案保护”。每周定时审查一次申请。最先成功申请技术方案保护的小组可以获得技术加分。技术方案保护需要公开，并共享给所有团队。